Course Name: (PCCCS495) Advanced Programming (OOP) Lab

Project Title: Smart Tourist Booking System

Student Name: Aadarsh Kumar Jha

Roll Number: 65

Instructor Name: Soumadip Biswas

Semester: Spring 2026

2. Problem Statement

In the tourism industry, managing bookings manually is often inefficient, time-consuming, and prone to errors. Traditional systems lack proper organization, leading to issues such as duplicate bookings, incorrect data handling, and difficulty in maintaining user and destination records. These limitations reduce overall system reliability and user satisfaction.
The aim of this project is to develop a **Smart Tourist Booking System** using Java and Object-Oriented Programming (OOP) principles. The system is designed to provide a structured and automated solution for managing tourist bookings, user information, and destination details.
The application allows users to create bookings, view available tourist spots, and manage booking records efficiently. By implementing core OOP concepts such as encapsulation, abstraction, and modular design, the system ensures better code organization, scalability, and maintainability.
Overall, this project aims to improve efficiency, reduce manual errors, and provide a reliable and user-friendly booking system.

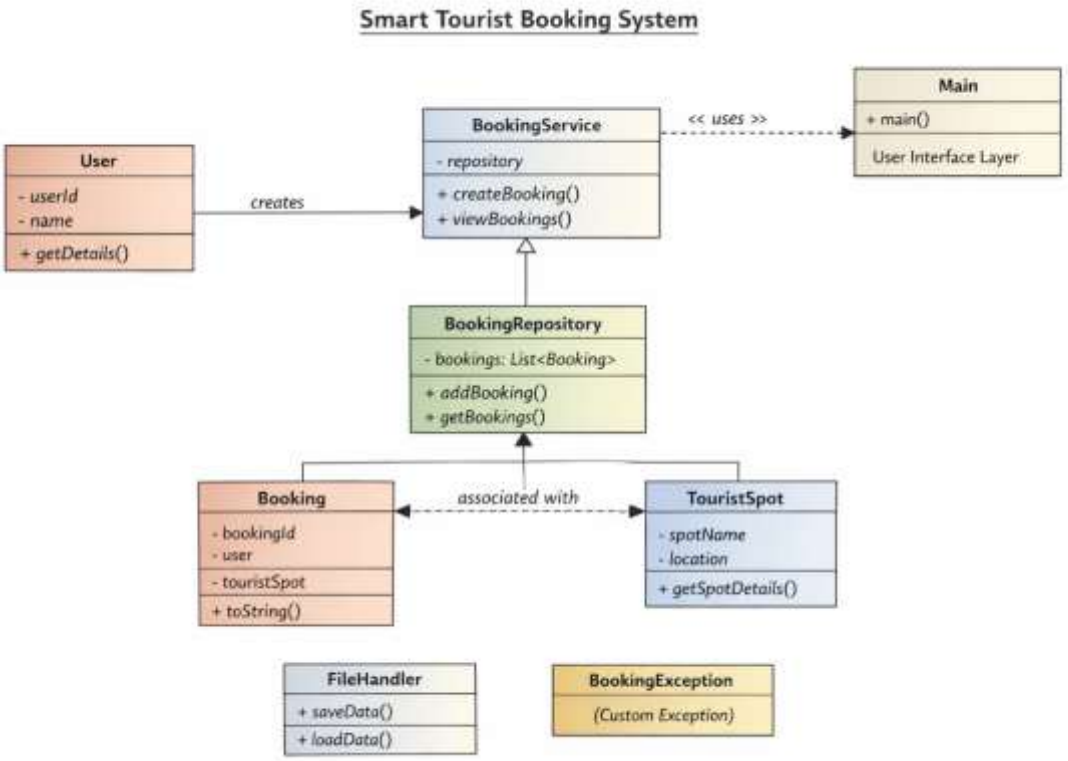
Key Objectives

- To design and develop a tourist booking system using Java
- To implement Object-Oriented Programming concepts such as encapsulation and abstraction
- To provide functionality for creating and managing bookings
- To maintain and manage user and tourist spot information efficiently
- To implement proper exception handling for invalid inputs
- To ensure modular and well-structured code using packages
- To improve system reliability and reduce manual errors
- To build a scalable system that can be enhanced in the future

3. System Design

The Smart Tourist Booking System is designed using a layered and modular architecture based on Object-Oriented Programming principles. The system is divided into multiple components to ensure separation of concerns, maintainability, and scalability.

3.1 UML Class Diagram (Conceptual Description)



The system consists of the following main classes:

- User
- Booking
- TouristSpot
- BookingService
- BookingRepository
- FileHandler
- BookingException
- UserFactory
- Main

Relationships:

- A User can create multiple Bookings
- A Booking is associated with a TouristSpot
- The Main class interacts with BookingService
- The BookingService communicates with BookingRepository
- The Repository manages data objects such as Booking, User, and TouristSpot
- Utility and exception classes support system operations

3.2 Description of Major Classes

User Class

Stores user information such as ID and name. Provides getter and setter methods for controlled access.

TouristSpot Class

Represents tourist destinations with attributes such as name and description.

Booking Class

Represents a booking record that links a user with a tourist spot.

BookingService Class

Handles core business logic such as creating bookings, viewing bookings, and validating inputs.
BookingRepository Class
Responsible for storing and retrieving booking data. Acts as a data access layer.
FileHandler Class
Handles file operations such as saving and loading booking data for persistence.
UserFactory Class
Provides a structured way to create user objects.
BookingException Class
Custom exception used to handle invalid inputs and booking-related errors.
Main Class
Acts as the entry point of the application and provides a menu-driven interface for user interaction.

3.3 Package Structure

The project is organized into the following packages:

- main → Contains Main.java (entry point)
- model → Contains User, Booking, TouristSpot
- service → Contains BookingService
- repository → Contains BookingRepository
- util → Contains FileHandler, UserFactory
- exception → Contains BookingException

This structure improves modularity and separates different responsibilities**.**

3.4 Interaction Flow

1. The application starts from the Main class
2. The user selects an option from the menu (e.g., create booking, view booking)
3. The Main class sends the request to the BookingService
4. The Service layer processes the logic and validates input
5. The Repository layer stores or retrieves data
6. Utility classes handle additional tasks like file operations
7. The result is returned to the Main class
8. Output is displayed to the user

3.5 Design Approach

- Layered architecture (Main → Service → Repository → Model)
- Separation of concerns for better maintainability
- Use of OOP principles such as encapsulation and abstraction
- Scalable design for future enhancements

4. OOP Concepts Demonstrated

1. Encapsulation

Where it is used:
In User and Booking classes to protect data.

Code Snippet:

```
private String name;

public void setName(String name) {
    this.name = name;
}

public String getName() {
    return name;
}
```

Why it is used:
To restrict direct access to variables and ensure controlled modification.

Explanation:
The data members are declared private and accessed through public getter and setter methods, which protects data integrity.

---

2. Abstraction

Where it is used:
In BookingService class to hide complex business logic.

Code Snippet:

```
public void viewBookings() {
    List<Booking> bookings = repository.getBookings();
    for (Booking b : bookings) {
        System.out.println(b);
    }
}
```

Why it is used:
To hide internal implementation and expose only essential functionalities.

Explanation:
The user interacts with service methods without knowing how data is stored or processed internally.

---

3. Separation of Concerns (Modularity)

Where it is used:
In package structure: model, service, repository, util, exception.

Code Snippet:

```
// service layer
BookingService service = new BookingService();
// repository layer
repository.addBooking(booking);
```

Why it is used:
To divide the system into independent modules for better maintainability.

Explanation:
Each layer has a specific responsibility, reducing complexity and making the system easier to manage and extend.

---

4. Polymorphism

Where it is used:

In method overriding (e.g., toString() in Booking class).

Code Snippet:

```
@Override
public String toString() {
   return "Booking for " + touristSpot.getSpotName();
}
```

Why it is used:

To provide different implementations of a common method.

Explanation:

The toString() method is overridden to display meaningful booking details instead of default object representation.

.

5. Implementation Highlights

5.1 Booking Creation with Validation

```
public void createBooking(User user, TouristSpot spot) throws BookingException {
    if (user == null || spot == null) {
        throw new BookingException("Invalid booking details");
    }
    Booking booking = new Booking(user, spot);
    repository.addBooking(booking);
}
```

Justification: Prevents invalid bookings and ensures only correct data is stored.

5.2 Viewing Bookings

```
List<Booking> bookings = repository.getBookings();
for (Booking b : bookings) {
    System.out.println(b);
}
```

Justification: Allows users to view all bookings in a structured manner.

5.3 Custom Exception Handling

```
if (user == null) {
    throw new BookingException("User not found");
}
```

Justification: Handles errors gracefully and prevents system crashes.

5.4 Data Storage using Collections

```
List<Booking> bookings = new ArrayList<>();
bookings.add(booking);
```

Justification: Enables dynamic storage and efficient management of booking data.

5.5 File Handling (Persistence)

```
writer.println(user.getName() + "," + spot.getSpotName());
```

Justification: Stores booking data in a simple format for later retrieval.

6. Testing & Error Handling

1. Test Cases

- Create booking successfully
  - View all bookings
  - Invalid booking → exception
  - Retrieve stored booking data

2. Edge Cases

- No bookings available → displays empty list
  - Null user or tourist spot → handled using exception
  - Empty data storage → system still runs without crash

3. Failure Scenarios

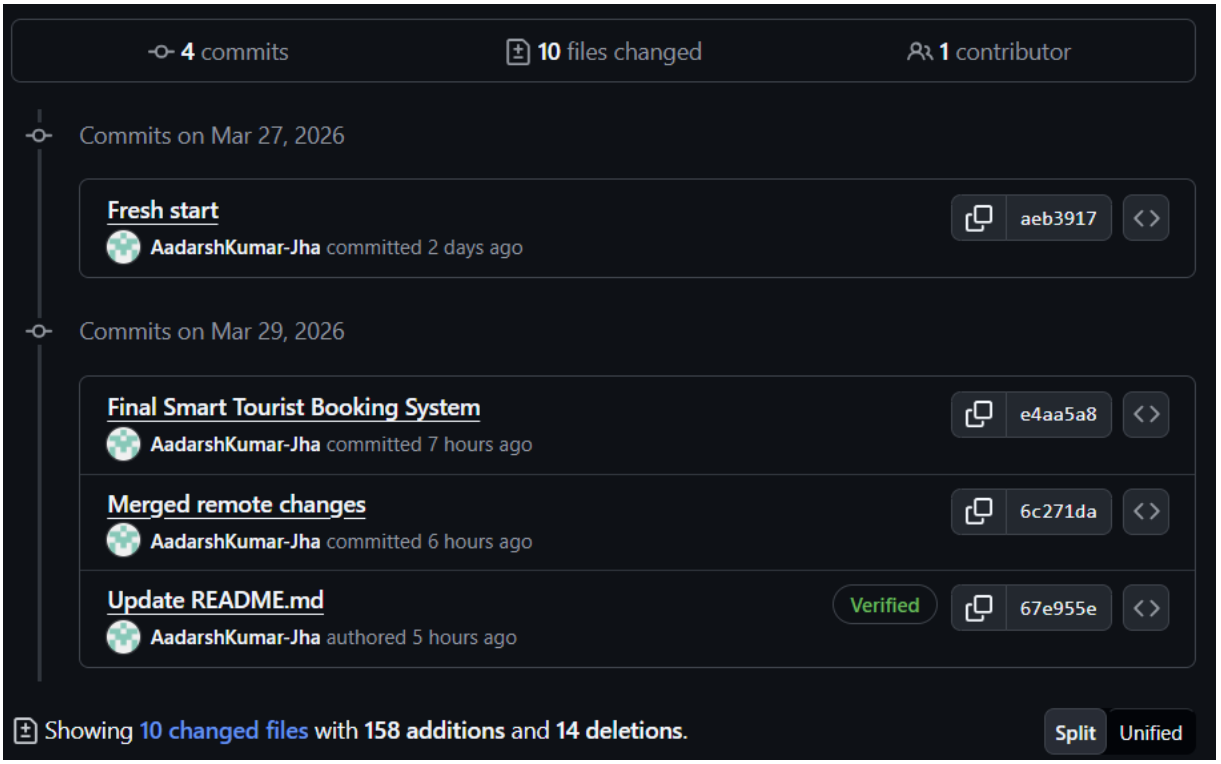
- Invalid booking details → BookingException
  - User not found → BookingException
  - Null input → handled safely with validation

4. Identified Limitation

- No restriction on duplicate bookings for same user and spot

7. Git Workflow

Commit History Screenshot



Explanation of Development Progression

The project was developed incrementally using Git, with each major feature implemented and committed separately.

---

1. Initial Setup

- Repository initialized using GitHub Classroom
- Basic project structure created with folders:
  - ❖ src
  - ❖ model
  - ❖ service
  - ❖ repository
  - ❖ util
- Initial project skeleton and setup done

Commit:

"Fresh start"

---

2. Core System Implementation

Added core source code for:

❖ User management (User.java)

❖ Tourist spot management (TouristSpot.java)

❖ Booking system (Booking.java)

❖ Service layer (BookingService.java)

❖ Repository handling (BookingRepository.java)

Commit:

"Final Smart Tourist Booking System"

---

3. Feature Integration & System Completion
● **Implemented main functionalities:**
○ Add booking
○ View bookings
○ Manage users
○ Handle tourist spots
● **Integrated all layers (model, service, repository)**
● **Ensured smooth interaction between components**
Commit:
"Final Smart Tourist Booking System"

---

4. Synchronization with Remote Repository
● Pulled latest changes from GitHub
● Merged remote updates with local repository
● Resolved conflicts (if any)
Commit:
"Merged remote changes"

---

5. Documentation Update
● **Updated README.md with:**
○ Project title
○ Description
○ Features
○ Usage instructions
Commit:
"Update README.md"

---

6. Final Refinement
● Minor fixes and improvements
● Cleaned up code and ensured stability
● Prepared final version for submission
Commit:
"Update README.md"

Git was used to track the development of the project in a structured manner. Each feature was implemented incrementally and committed separately, ensuring better version control, easier debugging, and clear project progression.

8. Conclusion

The **Smart Tourist Booking System** successfully automates the process of managing tourist bookings using Object-Oriented Programming principles, file handling, and exception management. The system provides an organized way to handle users, tourist spots, and bookings efficiently.

By eliminating manual processes, the system reduces errors, improves data management, and ensures a smoother booking experience. The modular design using packages like model, service, repository, and util enhances maintainability and scalability of the application.

Future Scope

● Implementation of a **Graphical User Interface (GUI)** or web-based application for better user experience
● Integration with a **database (MySQL/SQLite)** for efficient and secure data storage
● Addition of **multi-user support** with authentication and authorization
● Integration of **real-time booking updates and notifications**
● Implementation of **payment gateway system** for complete booking functionality
● Enhancement of validation to **prevent duplicate bookings and improve data integrity**
● Introduction of **unit testing (JUnit)** to improve reliability and maintain code quality

---

Appendix

Confirmation Mail of Project Minor

S soumadip.biswas@ieminternational.ai

Reply Reply all Forward

To: Aadarsh KumarJha

Tue 3/10/2026 3:50 PM

This message was identified as junk. We'll delete it after 11 days.

It's not junk

Retention: Junk Email (30 days) Expires: Thu 4/9/2026 3:50 PM

Dear Aadarsh Kumar Jha ,

Project: Smart Tourist Spot Ticket Booking and Crowd Regulation System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal presents a clear and highly relevant problem statement regarding overcrowding at tourist spots and offers a well-articulated solution through a Smart Ticket Booking and Crowd Regulation System. Key strengths include:
1. **Problem-Solution Alignment:** The proposed features, particularly time-slot booking and automatic capacity control, directly address the core problem of overcrowding and inefficient management.
2. **Robust Feature Set:** The outlined features are comprehensive, covering essential aspects like user registration, booking, cancellation, persistent storage, and exception handling, indicating a complete functional scope for a university project.
3. **Solid Architectural Design:** The adoption of a layered object-oriented architecture (Model, Service, Persistence) demonstrates a strong understanding of modularity, separation of concerns, and maintainability, which are critical for robust OOP development.
4. **Appropriate OOP Concepts:** The proposal correctly identifies and plans to utilize fundamental OOP principles (Abstraction, Encapsulation, Inheritance, Polymorphism, Collections, Exception Handling), providing a strong conceptual foundation.
5. **Focus on Crowd Regulation:** The explicit emphasis on automatic slot capacity control is a core differentiator and a crucial functional requirement that tackles the problem effectively.

Improvements:
To further strengthen the proposal and demonstrate deeper design considerations, the following improvements are suggested:
1. **Specify Design Pattern(s):** While 'Design Pattern' is listed, explicitly naming and briefly describing which specific pattern(s) (e.g., Factory for object creation, Strategy for different validation rules, or Repository for persistence) will be used and how they apply would add significant value and clarity.
2. **Detail Persistence Mechanism:** Provide a brief description of *how* the file handling for persistent storage will be implemented (e.g., using Java Serialization for objects, storing data in CSV/JSON format, or a custom binary format). This would show more thought into the implementation details of the Persistence layer.
3. **Consider Concurrency (Optional for basic course):** Acknowledge the challenges of concurrent booking in a real-world system. While full implementation of multi-threading might be out of scope for a basic OOP course, even a brief mention of potential considerations (e.g., synchronized blocks for critical sections) demonstrates foresight and awareness of real-world system complexities.

Regards,

Project Proposal Submission

Smart Tourist Spot Ticket Booking and Crowd Management System

Problem Statement

Popular tourist destinations often experience overcrowding during weekends, holidays, and peak travel seasons. Traditional ticketing systems usually rely on manual counters or simple booking methods that do not regulate the number of visitors entering at a specific time. This results in long queues, overcrowded spaces, safety concerns, and a poor overall experience for tourists. The proposed Smart Tourist Spot Ticket Booking and Crowd Management System allows tourists to book tickets online for specific attractions and time slots. The system maintains a fixed capacity for each slot and prevents bookings once the limit is reached. Each booking stores details such as tourist information, attraction name, and visit time. Using an object-oriented structure, the system ensures efficient crowd regulation and organized ticket management.

Target User

Tourists visiting popular attractions and administrators responsible for managing tourist entry and crowd control.

Core Features

- Browse available tourist attractions
- Register tourist details
- Book tickets for selected attractions and time slots
- Automatic slot capacity control to prevent overcrowding
- View and cancel bookings
- Persistent storage of booking data
- Exception handling for invalid operations

OOP Concepts to be Used

- Abstraction – Abstract User class defining common attributes and behavior
- Inheritance – Tourist and Admin classes extend the User class
- Polymorphism – Methods like displayDetails() overridden in subclasses
- Encapsulation – Private variables accessed through getters and setters
- Exception Handling – Handling errors such as full slots or invalid booking IDs
- Collections – ArrayList and HashMap used for storing attractions and bookings
- Design Pattern – Singleton pattern for BookingManager

Architecture